

# El problema de la satisfacibilidad booleana y el vacío de la intersección con lenguajes regulares

Jossué Arteché Muñoz

Facultad de Matemática y Computación  
Universidad de La Habana

10 de junio del 2026

Tutores: MSc. Fernando Raul Rodriguez Flores  
Lic. Raudel Alejandro Gómez Molina

# Problema de la satisfacibilidad booleana (SAT)

- Determinar si una fórmula booleana es satisfacible.

# Problema de la satisfacibilidad booleana (SAT)

- Determinar si una fórmula booleana es satisfacible.
- Primer problema demostrado NP-completo.

# Problema de la satisfacibilidad booleana (SAT)

- Determinar si una fórmula booleana es satisfacible.
- Primer problema demostrado NP-completo.
- Aplicaciones:
  - Verificación formal de hardware y software.
  - Planificación y calendarización.
  - Inteligencia artificial y razonamiento automático.
  - Diseño y optimización de circuitos.

# Problema de la satisfacibilidad booleana (SAT)

- Determinar si una fórmula booleana es satisfacible.
- Primer problema demostrado NP-completo.
- Aplicaciones:
  - Verificación formal de hardware y software.
  - Planificación y calendarización.
  - Inteligencia artificial y razonamiento automático.
  - Diseño y optimización de circuitos.
- Resolver SAT eficientemente implica demostrar  $P = NP$ .

# Enfoque basado en lenguajes formales

- En MATCOM se estudian enfoques basados en lenguajes formales.

# Enfoque basado en lenguajes formales

- En MATCOM se estudian enfoques basados en lenguajes formales.
- A partir de una instancia de SAT se construye un DFA.

# Enfoque basado en lenguajes formales

- En MATCOM se estudian enfoques basados en lenguajes formales.
- A partir de una instancia de SAT se construye un DFA.
- Se hacen consistentes las interpretaciones con formalismos:

## **Gramáticas Libres de Contexto**

- Se determina si es vacía la intersección.

## **Gramáticas Matriciales**

## **Gramáticas de Concatenación de Rango**



# Enfoque basado en lenguajes formales

Si existiera un formalismo que:

# Enfoque basado en lenguajes formales

Si existiera un formalismo que:

- Puede intersectarse eficientemente con un DFA.

Si existiera un formalismo que:

- Puede intersectarse eficientemente con un DFA.
- Tiene problema del vacío decidible en tiempo polinomial.

Si existiera un formalismo que:

- Puede intersectarse eficientemente con un DFA.
- Tiene problema del vacío decidible en tiempo polinomial.

Entonces SAT podría resolverse en tiempo polinomial.

## Fundamental Study

---

### On multiple context-free grammars\*

Hiroyuki Seki

*Department of Information and Computer Sciences, Faculty of Engineering Science,  
Osaka University, Toyonaka, Osaka 560, Japan*

Takashi Matsumura

*Nomura Research Institute, Ltd., Tyu-o-ku, Tokyo 104, Japan*

Mamoru Fujii

*College of General Education, Osaka University, Toyonaka, Osaka, 560, Japan*

Tadao Kasami

*Department of Information and Computer Sciences, Faculty of Engineering Science,  
Osaka University, Toyonaka, Osaka 560, Japan*

## Fundamental Study

---

### On multiple context-free grammars\*

Se definen las **gramáticas libres de contexto múltiple paralelas** (pMCFG).

Hiroyuki Seki

*Department of Information and Computer Sciences, Faculty of Engineering Science,  
Osaka University, Toyonaka, Osaka 560, Japan*

Takashi Matsumura

*Nomura Research Institute, Ltd., Tyu-o-ku, Tokyo 104, Japan*

Mamoru Fujii

*College of General Education, Osaka University, Toyonaka, Osaka, 560, Japan*

Tadao Kasami

*Department of Information and Computer Sciences, Faculty of Engineering Science,  
Osaka University, Toyonaka, Osaka 560, Japan*

## Fundamental Study

---

### On multiple context-free grammars\*

Hiroyuki Seki

*Department of Information and Computer Sciences, Faculty of Engineering Science,  
Osaka University, Toyonaka, Osaka 560, Japan*

Takashi Matsumura

*Nomura Research Institute, Ltd., Tyu-o-ku, Tokyo 104, Japan*

Mamoru Fujii

*College of General Education, Osaka University, Toyonaka, Osaka, 560, Japan*

Tadao Kasami

*Department of Information and Computer Sciences, Faculty of Engineering Science,  
Osaka University, Toyonaka, Osaka 560, Japan*

Se definen las **gramáticas libres de contexto múltiple paralelas** (pMCFG).

- Se pueden intersectar con un DFA.

## Fundamental Study

---

### On multiple context-free grammars\*

Hiroyuki Seki

*Department of Information and Computer Sciences, Faculty of Engineering Science,  
Osaka University, Toyonaka, Osaka 560, Japan*

Takashi Matsumura

*Nomura Research Institute, Ltd., Tyu-o-ku, Tokyo 104, Japan*

Mamoru Fujii

*College of General Education, Osaka University, Toyonaka, Osaka, 560, Japan*

Tadao Kasami

*Department of Information and Computer Sciences, Faculty of Engineering Science,  
Osaka University, Toyonaka, Osaka 560, Japan*

Se definen las **gramáticas libres de contexto múltiple paralelas** (pMCFG).

- Se pueden intersectar con un DFA.
- Tienen vacío decidible en tiempo polinomial.



# Por tanto

Por tanto

$$P = NP$$

# Por tanto

$$P = NP$$

...pero eso no es lo que vamos a ver hoy.

# Objetivos del trabajo

# Objetivos del trabajo

## Objetivo 1

Formular un teorema que caracterice los formalismos capaces de describir interpretaciones satisfacibles.

# Objetivos del trabajo

## Objetivo 1

Formular un teorema que caracterice los formalismos capaces de describir interpretaciones satisfacibles.

## Objetivo 2

Analizar críticamente las propiedades de las pMCFG propuestas en la literatura.

# Estructura de la presentación

## 1 Preliminares

- Teoría de lenguajes
- Definición de SAT

## 2 Construcción del lenguaje de las interpretaciones satisfacibles

- Codificación de las interpretaciones como cadenas
- Autómata fórmula
- Recuperar la consistencia de las interpretaciones





- **Alfabeto** ( $\Sigma$ ): conjunto finito de símbolos.

$$\Sigma = \{a, b, c\}$$

- **Alfabeto** ( $\Sigma$ ): conjunto finito de símbolos.

$$\Sigma = \{a, b, c\}$$

- **Cadena**: secuencia finita de símbolos de  $\Sigma$ .

$$abca \in \Sigma^* \quad \varepsilon \text{ (cadena vacía)}$$

- **Alfabeto** ( $\Sigma$ ): conjunto finito de símbolos.

$$\Sigma = \{a, b, c\}$$

- **Cadena**: secuencia finita de símbolos de  $\Sigma$ .

$$abca \in \Sigma^* \quad \varepsilon \text{ (cadena vacía)}$$

- **Concatenación**:

$$\alpha = ab \text{ y } \beta = ca \implies \alpha\beta = abca$$

- **Alfabeto** ( $\Sigma$ ): conjunto finito de símbolos.

$$\Sigma = \{a, b, c\}$$

- **Cadena**: secuencia finita de símbolos de  $\Sigma$ .

$$abca \in \Sigma^* \quad \varepsilon \text{ (cadena vacía)}$$

- **Concatenación**:

$$\alpha = ab \text{ y } \beta = ca \implies \alpha\beta = abca$$

- **Lenguaje**: conjunto de cadenas.

$$L = \{a^n b^n \mid n \geq 0\}$$

- **Alfabeto** ( $\Sigma$ ): conjunto finito de símbolos.

$$\Sigma = \{a, b, c\}$$

- **Cadena**: secuencia finita de símbolos de  $\Sigma$ .

$$abca \in \Sigma^* \quad \varepsilon \text{ (cadena vacía)}$$

- **Concatenación**:

$$\alpha = ab \text{ y } \beta = ca \implies \alpha\beta = abca$$

- **Lenguaje**: conjunto de cadenas.

$$L = \{a^n b^n \mid n \geq 0\}$$

- **Formalismo**: mecanismo de algún tipo para describir lenguajes.

## Definición

Una gramática es un formalismo que genera cadenas mediante la aplicación de reglas de producción.

## Definición

Una gramática es un formalismo que genera cadenas mediante la aplicación de reglas de producción.

$$G = (N, \Sigma, P, S)$$

- $N$ : no terminales
- $\Sigma$ : terminales
- $P$ : producciones
- $S$ : símbolo inicial

## Definición

Una gramática es un formalismo que genera cadenas mediante la aplicación de reglas de producción.

$$G = (N, \Sigma, P, S)$$

- $N$ : no terminales
  - $\Sigma$ : terminales
  - $P$ : producciones
  - $S$ : símbolo inicial
- 

## Ejemplo

$$S \rightarrow aS$$

$$S \Rightarrow aS \Rightarrow aaS \Rightarrow aab$$

$$S \rightarrow b$$



## Definición

Una gramática es un formalismo que genera cadenas mediante la aplicación de reglas de producción.

$$G = (N, \Sigma, P, S)$$

- $N$ : no terminales
- $\Sigma$ : terminales
- $P$ : producciones
- $S$ : símbolo inicial

---

## Ejemplo

$$S \rightarrow aS$$

$$S \Rightarrow aS \Rightarrow aaS \Rightarrow aab$$

$$S \rightarrow b$$

$$L(A) = \{a^n b \mid n \geq 0\}.$$

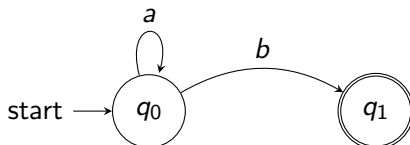
# Autómatas Finitos Deterministas (DFA)

# Autómatas Finitos Deterministas (DFA)

**Definición.** Un DFA es un modelo computacional que reconoce cadenas recorriendo estados de acuerdo con los símbolos leídos.

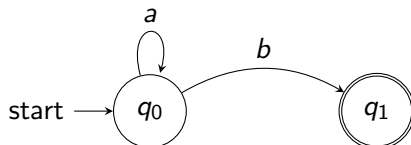
# Autómatas Finitos Deterministas (DFA)

**Definición.** Un DFA es un modelo computacional que reconoce cadenas recorriendo estados de acuerdo con los símbolos leídos.



# Autómatas Finitos Deterministas (DFA)

**Definición.** Un DFA es un modelo computacional que reconoce cadenas recorriendo estados de acuerdo con los símbolos leídos.

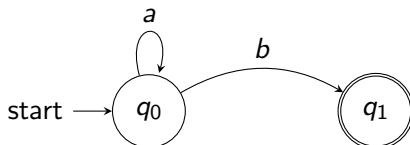


Cadenas aceptadas:

$b, ab, aab, aaab, \dots$

# Autómatas Finitos Deterministas (DFA)

**Definición.** Un DFA es un modelo computacional que reconoce cadenas recorriendo estados de acuerdo con los símbolos leídos.



Cadenas aceptadas:

$b, ab, aab, aaab, \dots$

Por tanto,

$$L(A) = \{a^n b \mid n \geq 0\}.$$

# Problemas de interés

## Intersección con DFA

Dado una instancia de un formalismo  $G$  y un DFA  $\mathcal{A}$ , ¿es posible construir una nueva instancia del mismo formalismo que describa

$$L(G) \cap L(\mathcal{A})?$$



## Intersección con DFA

Dado una instancia de un formalismo  $G$  y un DFA  $\mathcal{A}$ , ¿es posible construir una nueva instancia del mismo formalismo que describa

$$L(G) \cap L(\mathcal{A})?$$

Si la respuesta es sí, entonces el formalismo es **cerrado bajo intersección con DFA**.

# Problemas de interés

## Intersección con DFA

Dado una instancia de un formalismo  $G$  y un DFA  $\mathcal{A}$ , ¿es posible construir una nueva instancia del mismo formalismo que describa

$$L(G) \cap L(\mathcal{A})?$$

Si la respuesta es sí, entonces el formalismo es **cerrado bajo intersección con DFA**.

## Problema del vacío

Dada una instancia de un formalismo  $G$ ,

$$L(G) = \emptyset?$$

## Intersección con DFA

Dado una instancia de un formalismo  $G$  y un DFA  $\mathcal{A}$ , ¿es posible construir una nueva instancia del mismo formalismo que describa

$$L(G) \cap L(\mathcal{A})?$$

Si la respuesta es sí, entonces el formalismo es **cerrado bajo intersección con DFA**.

## Problema del vacío

Dada una instancia de un formalismo  $G$ ,

$$L(G) = \emptyset?$$

Determinar si el lenguaje generado contiene al menos una cadena.

## 1 Preliminares

- Teoría de lenguajes
- Definición de SAT

## 2 Construcción del lenguaje de las interpretaciones satisfacibles

- Codificación de las interpretaciones como cadenas
- Autómata fórmula
- Recuperar la consistencia de las interpretaciones

# Problema de la satisfacibilidad booleana (SAT)

- **Variable booleana:**  $x_1, x_2, x_3, \dots$  toman valores 1 (verdadero) o 0 (falso).

# Problema de la satisfacibilidad booleana (SAT)

- **Variable booleana:**  $x_1, x_2, x_3, \dots$  toman valores 1 (verdadero) o 0 (falso).
- **Literal:**  $x_i, \neg x_i$  (la variable o su negación).

# Problema de la satisfacibilidad booleana (SAT)

- **Variable booleana:**  $x_1, x_2, x_3, \dots$  toman valores 1 (verdadero) o 0 (falso).
- **Literal:**  $x_i, \neg x_i$  (la variable o su negación).
- **Cláusula:** Disyunción de literales.

$$(x_1 \vee \neg x_2 \vee x_3)$$

# Problema de la satisfacibilidad booleana (SAT)

- **Variable booleana:**  $x_1, x_2, x_3, \dots$  toman valores 1 (verdadero) o 0 (falso).
- **Literal:**  $x_i, \neg x_i$  (la variable o su negación).
- **Cláusula:** Disyunción de literales.

$$(x_1 \vee \neg x_2 \vee x_3)$$

- **Forma Normal Conjuntiva (FNC)**

Conjunción de cláusulas.

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$$



# Problema de la satisfacibilidad booleana (SAT)

- **Variable booleana:**  $x_1, x_2, x_3, \dots$  toman valores 1 (verdadero) o 0 (falso).
- **Literal:**  $x_i, \neg x_i$  (la variable o su negación).
- **Cláusula:** Disyunción de literales.

$$(x_1 \vee \neg x_2 \vee x_3)$$

- **Forma Normal Conjuntiva (FNC)**

Conjunción de cláusulas.

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$$

## Problema SAT

¿Existe una asignación de valores de verdad para las variables que haga verdadera toda la fórmula?

## 1 Preliminares

- Teoría de lenguajes
- Definición de SAT

## 2 Construcción del lenguaje de las interpretaciones satisfacibles

- Codificación de las interpretaciones como cadenas
- Autómata fórmula
- Recuperar la consistencia de las interpretaciones

# Codificación de interpretaciones como cadenas

## Idea

# Codificación de interpretaciones como cadenas

## Idea

Cada asignación de variables se representa como una cadena.

# Codificación de interpretaciones como cadenas

## Idea

Cada asignación de variables se representa como una cadena.

## Regla de codificación

- Si la variable  $x_i = 1$ , el símbolo  $i$  es 1.

# Codificación de interpretaciones como cadenas

## Idea

Cada asignación de variables se representa como una cadena.

## Regla de codificación

- Si la variable  $x_i = 1$ , el símbolo  $i$  es 1.
- Si  $x_i = 0$ , el símbolo  $i$  es 0.

# Codificación de interpretaciones como cadenas

## Idea

Cada asignación de variables se representa como una cadena.

## Regla de codificación

- Si la variable  $x_i = 1$ , el símbolo  $i$  es 1.
- Si  $x_i = 0$ , el símbolo  $i$  es 0.
- Se añade el símbolo final  $d$ .

# Codificación de interpretaciones como cadenas

## Idea

Cada asignación de variables se representa como una cadena.

## Regla de codificación

- Si la variable  $x_i = 1$ , el símbolo  $i$  es 1.
- Si  $x_i = 0$ , el símbolo  $i$  es 0.
- Se añade el símbolo final  $d$ .
- Se repite la cadena por cada cláusula.



# Codificación de interpretaciones como cadenas

## Idea

Cada asignación de variables se representa como una cadena.

## Regla de codificación

- Si la variable  $x_i = 1$ , el símbolo  $i$  es 1.
- Si  $x_i = 0$ , el símbolo  $i$  es 0.
- Se añade el símbolo final  $d$ .
- Se repite la cadena por cada cláusula.

---

## Ejemplo (4 variables, 2 cláusulas)

# Codificación de interpretaciones como cadenas

## Idea

Cada asignación de variables se representa como una cadena.

## Regla de codificación

- Si la variable  $x_i = 1$ , el símbolo  $i$  es 1.
- Si  $x_i = 0$ , el símbolo  $i$  es 0.
- Se añade el símbolo final  $d$ .
- Se repite la cadena por cada cláusula.

---

## Ejemplo (4 variables, 2 cláusulas)

Asignación:

$$x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 0$$

# Codificación de interpretaciones como cadenas

## Idea

Cada asignación de variables se representa como una cadena.

## Regla de codificación

- Si la variable  $x_i = 1$ , el símbolo  $i$  es 1.
- Si  $x_i = 0$ , el símbolo  $i$  es 0.
- Se añade el símbolo final  $d$ .
- Se repite la cadena por cada cláusula.

---

## Ejemplo (4 variables, 2 cláusulas)

Asignación:

Cadena base:

$$x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 0$$

1010

# Codificación de interpretaciones como cadenas

## Idea

Cada asignación de variables se representa como una cadena.

## Regla de codificación

- Si la variable  $x_i = 1$ , el símbolo  $i$  es 1.
- Si  $x_i = 0$ , el símbolo  $i$  es 0.
- Se añade el símbolo final  $d$ .
- Se repite la cadena por cada cláusula.

---

## Ejemplo (4 variables, 2 cláusulas)

Asignación:

Cadena base:

$$x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 0$$

1010

Con separador:

1010 $d$

# Codificación de interpretaciones como cadenas

## Idea

Cada asignación de variables se representa como una cadena.

## Regla de codificación

- Si la variable  $x_i = 1$ , el símbolo  $i$  es 1.
- Si  $x_i = 0$ , el símbolo  $i$  es 0.
- Se añade el símbolo final  $d$ .
- Se repite la cadena por cada cláusula.

---

## Ejemplo (4 variables, 2 cláusulas)

Asignación:

$$x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 0$$

Con separador:

1010*d*

Cadena base:

1010

Como hay 2 cláusulas:

1010*d* 1010*d*

# Lenguaje de interpretaciones satisfacibles

## Idea

El conjunto de interpretaciones de una fórmula se puede representar como un lenguaje.

## Idea

El conjunto de interpretaciones de una fórmula se puede representar como un lenguaje.

## Definición intuitiva

$$L_{\text{sat}}F = \{ w \mid w \text{ codifica una interpretación que satisface } F \}$$



## Idea

El conjunto de interpretaciones de una fórmula se puede representar como un lenguaje.

## Definición intuitiva

$$L_{\text{sat}}F = \{ w \mid w \text{ codifica una interpretación que satisface } F \}$$

## Conexión con SAT

$$F \in SAT \iff L_{\text{sat}}F \neq \emptyset$$

## 1 Preliminares

- Teoría de lenguajes
- Definición de SAT

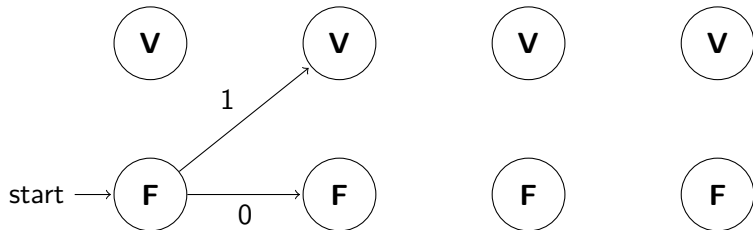
## 2 Construcción del lenguaje de las interpretaciones satisfacibles

- Codificación de las interpretaciones como cadenas
- Autómata fórmula
- Recuperar la consistencia de las interpretaciones

## Lenguaje de una cláusula:

$L_C = \{w \mid w \text{ codifica una interpretación que satisface } C\}$

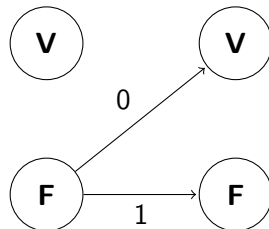
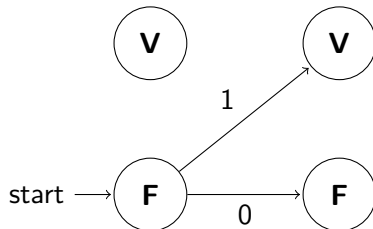
**Cláusula:**  $(x_1 \vee \neg x_3)$  puede verse como  $(x_1 \vee \cancel{x_2} \vee \neg x_3)$



## Lenguaje de una cláusula:

$L_C = \{w \mid w \text{ codifica una interpretación que satisface } C\}$

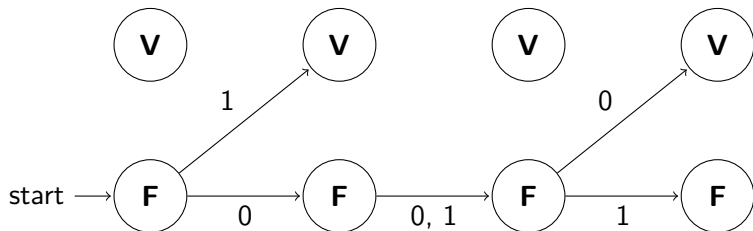
**Cláusula:**  $(x_1 \vee \neg x_3)$  puede verse como  $(x_1 \vee \cancel{x_2} \vee \neg x_3)$



## Lenguaje de una cláusula:

$L_C = \{w \mid w \text{ codifica una interpretación que satisface } C\}$

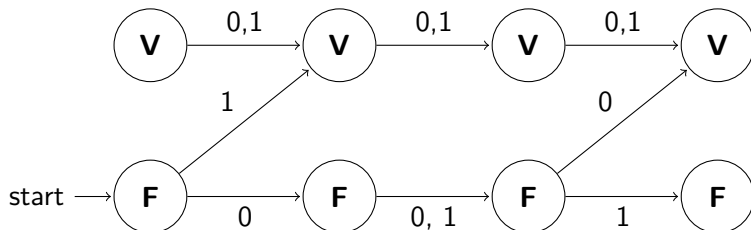
**Cláusula:**  $(x_1 \vee \neg x_3)$  puede verse como  $(x_1 \vee \cancel{x_2} \vee \neg x_3)$



## Lenguaje de una cláusula:

$L_C = \{w \mid w \text{ codifica una interpretación que satisface } C\}$

**Cláusula:**  $(x_1 \vee \neg x_3)$  puede verse como  $(x_1 \vee \cancel{x_2} \vee \neg x_3)$

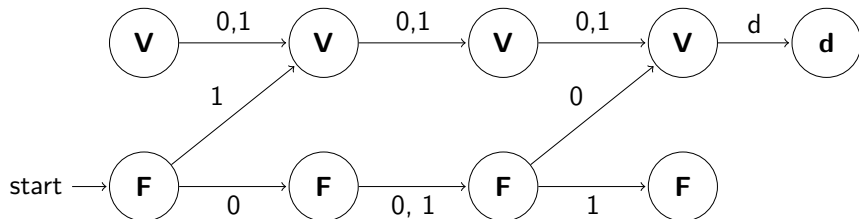


# Autómata cláusula con separador

## Lenguaje de una cláusula con separador:

$L_C = \{wd \mid w \text{ codifica una interpretación que satisface } C\}$

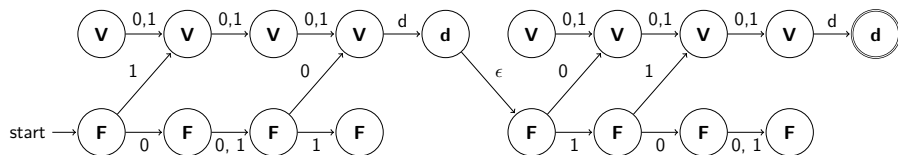
**Cláusula:**  $(x_1 \vee \neg x_3)$  puede verse como  $(x_1 \vee \cancel{x_2} \vee \neg x_3)$



## Lenguaje de interpretaciones independientes:

$L_{\text{ind}}(F) = \{w_1dw_2d \dots w_md \mid w_i \text{ es una interpretación que satisface } C_i\}$

**Fórmula:**  $(x_1 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2) \rightsquigarrow (x_1 \vee \cancel{x_2} \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \cancel{x_3})$

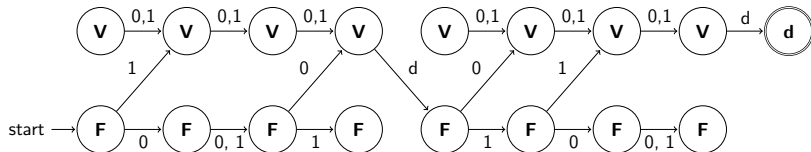




## Lenguaje de interpretaciones independientes:

$L_{\text{ind}}(F) = \{w_1dw_2d \dots w_md \mid w_i \text{ es una interpretación que satisface } C_i\}$

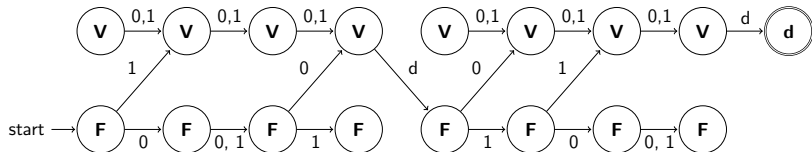
**Fórmula:**  $(x_1 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2) \rightsquigarrow (x_1 \vee \cancel{x_2} \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \cancel{x_3})$



## Lenguaje de interpretaciones independientes:

$L_{\text{ind}}(F) = \{w_1 d w_2 d \dots w_m d \mid w_i \text{ es una interpretación que satisface } C_i\}$

**Fórmula:**  $(x_1 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2) \rightsquigarrow (x_1 \vee \cancel{x_2} \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \cancel{x_3})$



Autómata fórmula  $\mathcal{A}_F$

$$L(\mathcal{A}_F) = L_{\text{ind}}(F)$$

## 1 Preliminares

- Teoría de lenguajes
- Definición de SAT

## 2 Construcción del lenguaje de las interpretaciones satisfacibles

- Codificación de las interpretaciones como cadenas
- Autómata fórmula
- Recuperar la consistencia de las interpretaciones